What we will learn

- ✓ Single Dimensional arrays
- ✓ Copying arrays
- ✓ Passing and returning array from method
- ✓ Searching and sorting arrays and the Array class
- ✓ Two-Dimensional array and its processing
- ✓ Passing Two-dimensional Array to methods
- ✓ Multidimensional Arrays

Why Array?

- Very often we need to deal with relatively large set of data.
- E.g.
  - Percentage of all the students of the college. (May be in thousands)
  - Age of all the citizens of the city. (May be lakhs)
- We need to declare thousands or lakhs of the variable to store the data which is practically not possible.
- We need a solution to store more data in a single variable.
- **Array** is the most appropriate way to handle such data.
- As per English Dictionary, "***Array means collection or group or arrangement in a specific order.***"

Array

- **An array is a fixed size sequential collection of elements of same data type grouped under single variable name.**

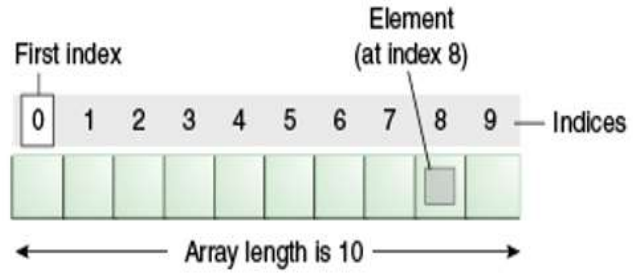
```
int percentage[10];
```

[0]	[1]	[2]	[3]	[4]	[5]	[6]	[7]	[8]	[9]

Fixed Size	Sequential	Same Data type	Single Variable Name
The size of an array is fixed at the time of declaration which cannot be changed later on. Here **array size** is **10**.	All the elements of an array are stored in a consecutive blocks in a memory. **10** (0 to 9)	Data type of all the elements of an array is same which is defined at the time of declaration. Here **data type** is **int**	All the elements of an array will be referred through common name. Here **array name** is **percentage**

Array declaration

- Normal Variable Declaration: int **a**;
- Array Variable Declaration: int **b**[10];
- Individual value or data stored in an array is known as an **element of an array**.
- Positioning / indexing of an elements in an array always **starts with 0 not 1**.
  - If 10 elements in an array then index is 0 to 9
  - If 100 elements in an array then index is 0 to 99
  - If 35 elements in an array then index is 0 to 34
- Variable **a** stores 1 integer number where as variable **b** stores 10 integer numbers which can be accessed as b[0], b[1], b[2], b[3], b[4], b[5], b[6], b[7], b[8] and b[9]



Array

- Important point about Java array.
  - An array is **derived** datatype.
  - An array is **dynamically** allocated.
  - The individual elements of an array is refereed by their **index/subscript** value.
  - The **subscript** for an array always begins with **0**.

35	13	28	106	35	42	5	83	97	14
a[0]	a[1]	a[2]	a[3]	a[4]	a[5]	a[6]	a[7]	a[8]	a[9]

One-Dimensional Array

- An array using **one subscript** to represent the **list of elements** is called **one dimensional array**.
- A One-dimensional array is essentially a **list** of **like-typed variables.**
- Array declaration: type var-name[];

  Example: int student_marks[];
- Above example will represent array with no value (null).
- To link **student_marks** with actual array of integers, we must allocate one using ***new*** keyword.

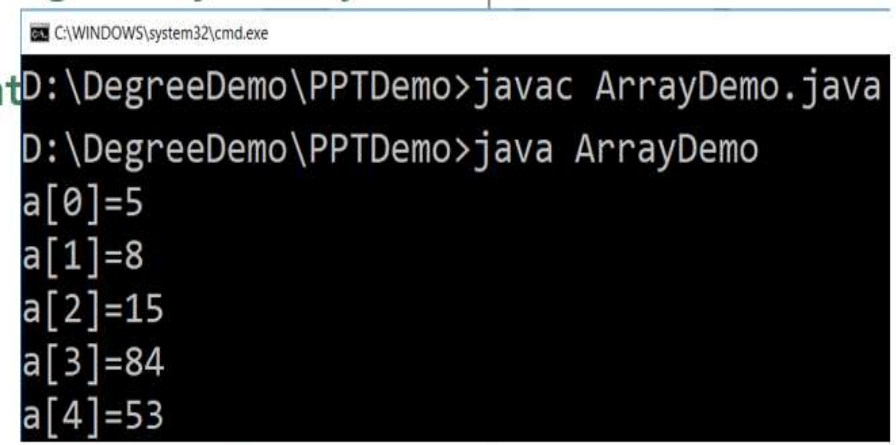
  Example: int student_marks[] = ***new*** int[20];

Example (Array)

```java
public class ArrayDemo{
    public static void main(String[] args) {
        int a[]; // or int[] a
        // till now it is null as it does not assigned any memory

        a = new int[5]; // here we actually creat
        a[0] = 5;
        a[1] = 8;
        a[2] = 15;
        a[3] = 84;
        a[4] = 53;

        /* in java we use length property to determine the length
         * of an array, unlike c where we used sizeof function */
        for (int i = 0; i < a.length; i++) {
                System.out.println("a["+i+"]="+a[i]);
        }
    }
```



WAP to store 5 numbers in an array and print them

```
1. import java.util.*;
2. class ArrayDemo1{
3. public static void main (String[] args){
4.       int i, n;
5.       int[] a=new int[5];
6.       Scanner sc = new Scanner(System.in);
7.       System.out.print("enter Array Length:");
8.       n = sc.nextInt();
9.       for(i=0; i<n; i++) {
10.           System.out.print("enter a["+i+"]:");
11.           a[i] = sc.nextInt();
12.      }
13.      for(i=0; i<n; i++)
14.          System.out.println(a[i]);
15.   }
16. }
```

Output:

```
enter Array
Length:5
enter a[0]:1
enter a[1]:2
enter a[2]:4
enter a[3]:5
enter a[4]:6
1
2
4
5
6
```

WAP to print elements of an array in reverse order

```
1.  import java.util.*;
2.  public class RevArray{
3.  public static void  main(String[] args) {
4.    int i, n;
5.    int[] a;
6.    Scanner sc=new Scanner(System.in);
7.    System.out.print("Enter Size of an Array:");
8.    n=sc.nextInt();
9.    a=new int[n];
10.   for(i=0; i<n; i++){
11.       System.out.print("enter a["+i+"]:");
12.       a[i]=sc.nextInt();
13.   }
14.   System.out.println("Reverse Array");
15.   for(i=n-1; i>=0; i--)
16.       System.out.println(a[i]);
17.  }
18. }
```

Output:

```
Enter Size of an
Array:5
enter a[0]:1
enter a[1]:2
enter a[2]:3
enter a[3]:4
enter a[4]:5
Reverse Array
5
4
3
2
1
```

WAP to count positive number, negative number and zero from an array of n size

```
import java.util.*;
class ArrayDemo1{
public static void main (String[] args){
      int n,pos=0,neg=0,z=0;
      int[] a=new int[5];
      Scanner sc = new Scanner(System.in);
      System.out.print("enter Array Length:");
      n = sc.nextInt();
      for(int i=0; i<n; i++) {
           System.out.print("enter a["+i+"]:");
           a[i] = sc.nextInt();
           if(a[i]>0)
             pos++;
           else if(a[i]<0)
             neg++;
           else
             z++;
      }
      System.out.println("Positive no="+pos);
      System.out.println("Negative no="+neg);
      System.out.println("Zero no="+z);
}}
```

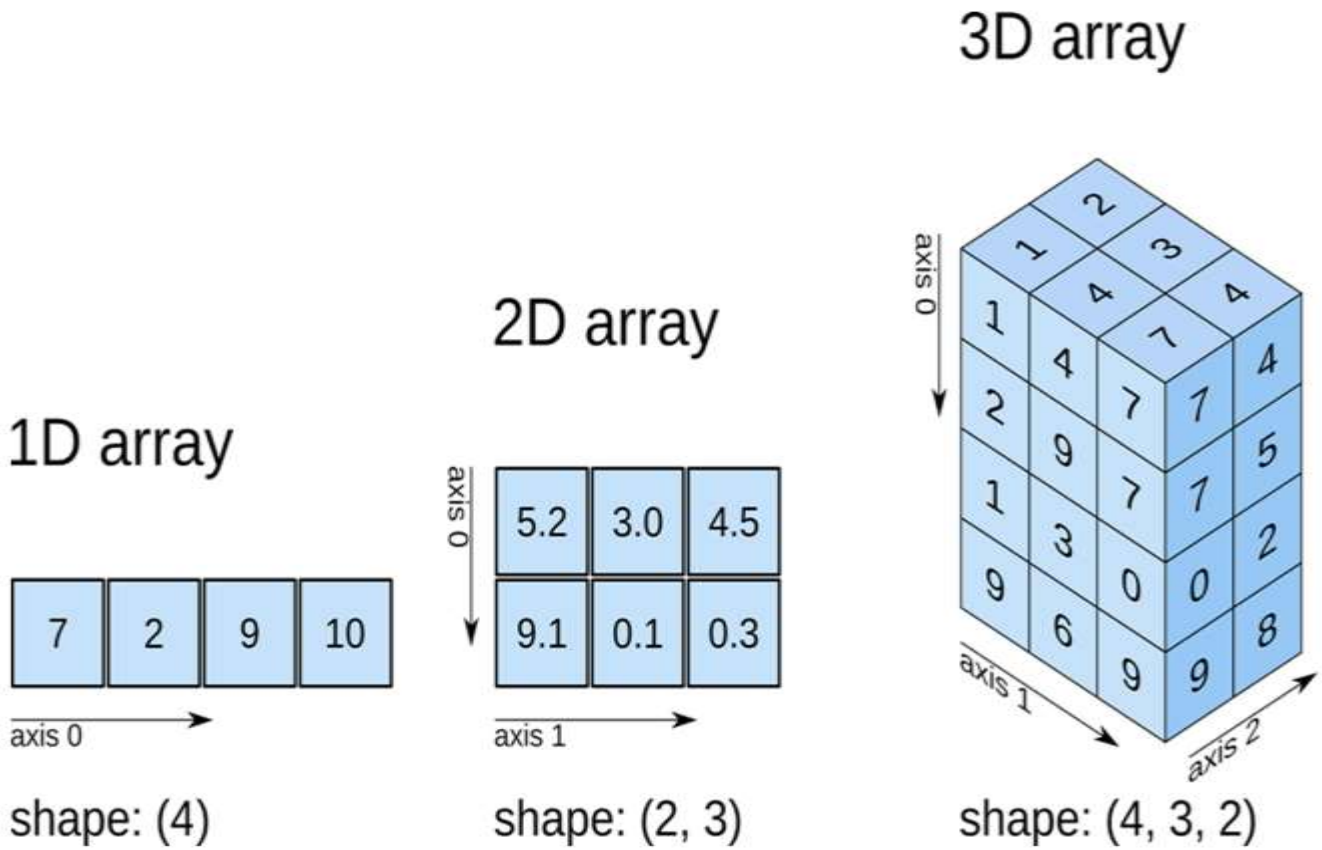
Output:

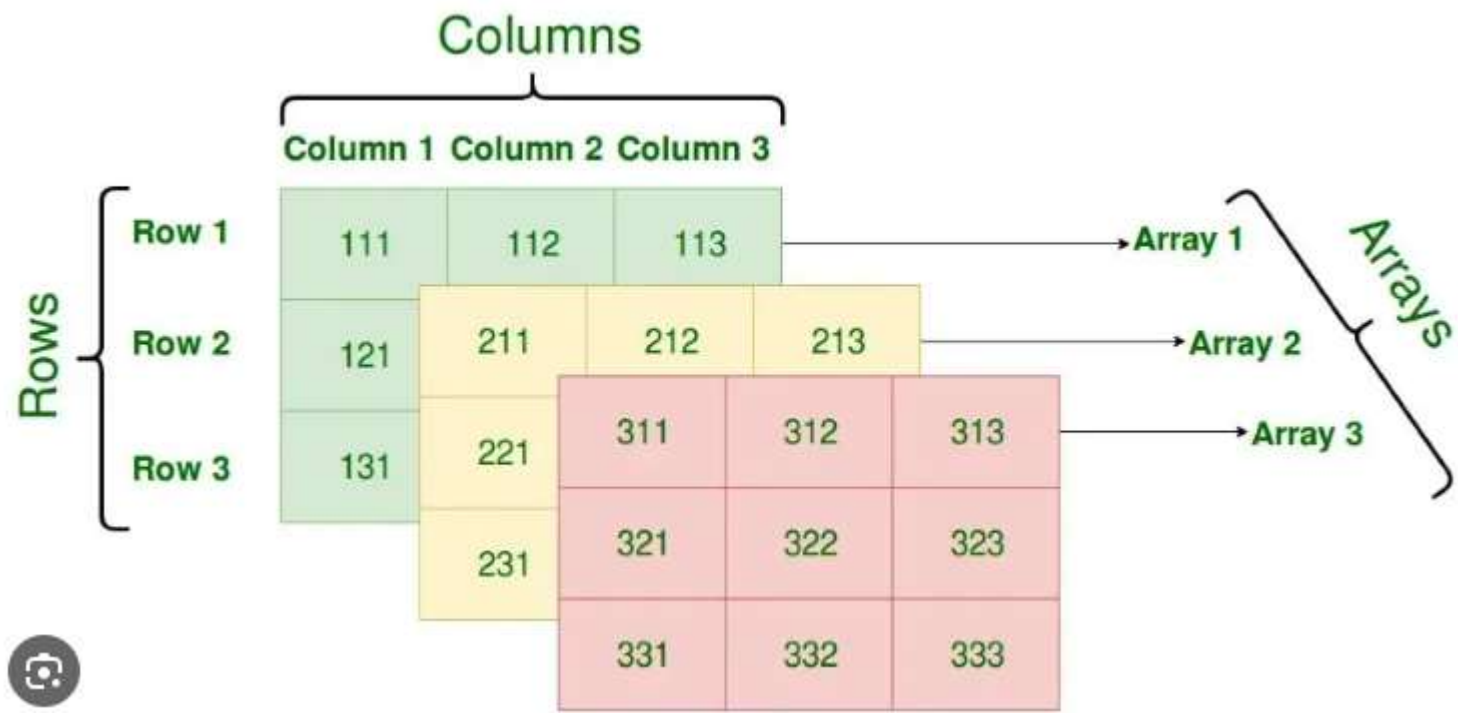
```
enter Array
Length:5
enter a[0]:-3
enter a[1]:5
enter a[2]:0
enter a[3]:-2
enter a[4]:00
Positive no=1
Negative no=2
Zero no=2
```

Exercise: Array

1. WAP to count odd and even elements of an array.
2. WAP to calculate sum and average of n numbers from an array.
3. WAP to find largest and smallest from an array.

Multidimensional Array




Columns
Column 1
Column 2
Column 3
Rows
Row 1
Row 2
Row 3
111
112
113
121
211
212
213
131
221
231
311
312
313
321
322
323
331
332
333
Array 1
Array 2
Array 3
Arrays

WAP to read 3 x 3 elements in 2d array

```
1. import java.util.*;
2. class Array2Demo{
3. public static void main(String[] args) {
4.    int size;
5.    Scanner sc=new Scanner(System.in);
6.    System.out.print("Enter size of an array");
7.    size=sc.nextInt();
8.    int a[][]=new int[size][size];
9.    for(int i=0;i<a.length;i++){
10.     for(int j=0;j<a.length;j++){
11.             a[i][j]=sc.nextInt();
12.     }
13.   }

14.   for(int i=0;i<a.length;i++){
15.     for(int j=0;j<a.length;j++){
16.          System.out.print("a["+i+"]["+j+"]:"+a[i][j]+"
17.     }
18.     System.out.println();
19.   }
20. }
21.}
```

	Column-0	Column-1	Column-2
Row-0	**11**	**18**	**-7**
Row-1	**25**	**100**	**0**
Row-2	**-4**	**50**	**88**

Output:

```
11
12
13
14
15
16
17
18
19
a[0][0]:11      a[0][1]:12      a[0][2]:13
a[1][0]:14      a[1][1]:15      a[1][2]:16
a[2][0]:17      a[2][1]:18      a[2][2]:19
```

WAP to perform addition of two 3 x 3 matrices

```
import java.util.*;
class Array2Demo{
public static void main(String[] args) {
int size;
int a[][],b[][],c[][];
Scanner sc=new Scanner(System.in);
 System.out.print("Enter size of an
                                array:");
 size=sc.nextInt();
 a=new int[size][size];
 System.out.println("Enter array
                                elements:");
for(int i=0;i<a.length;i++){
    for(int j=0;j<a.length;j++){
       System.out.print("Enter
                        a["+i+"]["+j+"]:");
       a[i][j]=sc.nextInt();
    }
}
```

```
b=new int[size][size];
for(int i=0;i<b.length;i++){
 for(int j=0;j<b.length;j++){
   System.out.print("Enter
                        b["+i+"]["+j+"]:");
   b[i][j]=sc.nextInt();
 }
}
c=new int[size][size];
for(int i=0;i<c.length;i++){
 for(int j=0;j<c.length;j++){
 System.out.print("c["+i+"]["+j+"]:“
                +(a[i][j]+b[i][j])+"\t");
  }
  System.out.println();
 }
 }//main()
}//class
```

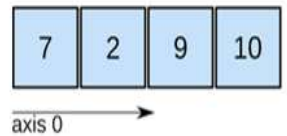
Output:

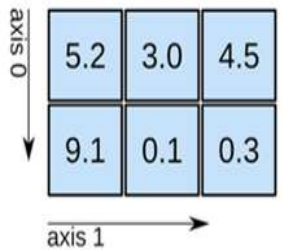
```
Enter size of an array:3
Enter array elements:
Enter a[0][0]:1
Enter a[0][1]:1
Enter a[0][2]:1
Enter a[1][0]:1
Enter a[1][1]:1
Enter a[1][2]:1
Enter a[2][0]:1
Enter a[2][1]:1
Enter a[2][2]:1
Enter b[0][0]:4
Enter b[0][1]:4
Enter b[0][2]:4
Enter b[1][0]:4
Enter b[1][1]:4
Enter b[1][2]:4
Enter b[2][0]:4
Enter b[2][1]:4
Enter b[2][2]:4
c[0][0]:5       c[0][1]:5       c[0][2]:5
c[1][0]:5       c[1][1]:5       c[1][2]:5
c[2][0]:5       c[2][1]:5       c[2][2]:5
```

Initialization of an array elements

1. One dimensional Array

1. `int a[5] = { 7, 3, -5, 0, 11 };` // a[0]=7, a[1] = 3, a[2] = -5, a[3] = 0, a[4] = 11
2. `int a[5] = { 7, 3 };` // a[0] = 7, a[1] = 3, a[2], a[3] and a[4] are 0
3. `int a[5] = { 0 };` // all elements of an array are initialized to 0





2. Two dimensional Array

1. `int a[2][4] = { { 7, 3, -5, 10 }, { 11, 13, -15, 2} };` // 1st row is 7, 3, -5, 10 & 2nd row is 11, 13, -15, 2
2. `int a[2][4] = { 7, 3, -5, 10, 11, 13, -15, 2 };` // 1st row is 7, 3, -5, 10 & 2nd row is 11, 13, -15, 2
3. `int a[2][4] = { { 7, 3 }, { 11} };` // 1st row is 7, 3, 0, 0 & 2nd row is 11, 0, 0, 0
4. `int a[2][4] = { 7, 3 };` // 1st row is 7, 3, 0, 0 & 2nd row is 0, 0, 0, 0
5. `int a[2][4] = { 0 };` // 1st row is 0, 0, 0, 0 & 2nd row is 0, 0, 0, 0

Multi-Dimensional Array

- In java, multidimensional array is actually **array of arrays.**
- **Example**: `int runPerOver[][] = new int[3][6];`

First Over (a[0])	**4** a[0][0]	**0** a[0][1]	**1** a[0][2]	**3** a[0][3]	**6** a[0][4]	**1** a[0][5]
Second Over (a[1])	**1** a[1][0]	**1** a[1][1]	**0** a[1][2]	**6** a[1][3]	**0** a[1][4]	**4** a[1][5]
Third Over (a[2])	**2** a[2][0]	**1** a[2][1]	**1** a[2][2]	**0** a[2][3]	**1** a[2][4]	**1** a[2][5]

- **`length` field:**
  - If we use length field with multidimensional array, it will return length of first dimension.
  - Here, if **runPerOver.length** is accessed it will return **3**
  - Also if **runPerOver[0].length** is accessed it will be **6**

Multi-Dimensional Array (Example)

```
Scanner s = new Scanner(System.in);
int runPerOver[][] = new int[3][6];
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 6; j++) {
        System.out.print("Enter Run taken" +
        " in Over numner " + (i + 1) +
        " and Ball number " + (j + 1) + " = ");
        runPerOver[i][j] = s.nextInt();
    }
}
int totalRun = 0;
for (int i = 0; i < 3; i++) {
    for (int j = 0; j < 6; j++) {
        totalRun += runPerOver[i][j];
    }
}
double average = totalRun / (double) runPerOver.length;
System.out.println("Total Run = " + totalRun);
System.out.println("Average per over = " + average);
```

```
C:\WINDOWS\system32\cmd.exe
Enter Run taken in Over numner 1 and Ball number 1 = 4
Enter Run taken in Over numner 1 and Ball number 2 = 0
Enter Run taken in Over numner 1 and Ball number 3 = 1
Enter Run taken in Over numner 1 and Ball number 4 = 3
Enter Run taken in Over numner 1 and Ball number 5 = 6
Enter Run taken in Over numner 1 and Ball number 6 = 1
Enter Run taken in Over numner 2 and Ball number 1 = 1
Enter Run taken in Over numner 2 and Ball number 2 = 1
Enter Run taken in Over numner 2 and Ball number 3 = 0
Enter Run taken in Over numner 2 and Ball number 4 = 6
Enter Run taken in Over numner 2 and Ball number 5 = 0
Enter Run taken in Over numner 2 and Ball number 6 = 4
Enter Run taken in Over numner 3 and Ball number 1 = 2
Enter Run taken in Over numner 3 and Ball number 2 = 1
Enter Run taken in Over numner 3 and Ball number 3 = 1
Enter Run taken in Over numner 3 and Ball number 4 = 0
Enter Run taken in Over numner 3 and Ball number 5 = 1
Enter Run taken in Over numner 3 and Ball number 6 = 1
Total Run = 33
Average per over = 11.0
```

Multi-Dimensional Array (Cont.)

- **manually** allocate **different** size:

```
int runPerOver[][] = new int[3][];
runPerOver[0] =  new int[6];
runPerOver[1]  =  new int[7];
runPerOver[2]  =  new int[6];
```

- **initialization**:

```
int runPerOver[][] = {
                        {0,4,2,1,0,6},
                        {1,-1,4,1,2,4,0},
                        {6,4,1,0,2,2},
                }
```

Note : here to specify extra runs (Wide, No Ball etc..) negative values are used

Searching in Array

- Searching is the process of looking for a specific element in an array. for example, discovering whether a certain element is included in the array.
- Searching is a common task in computer programming. Many algorithms and data structures are devoted to searching.
- We will discuss two commonly used approaches,
  - **Linear Search:** The linear search approach compares the key element key sequentially with each element in the array. It continues to do so until the key matches an element in the array or the array is exhausted without a match being found.
  - **Binary Search:** The binary search first compares the key with the element in the middle of the array. Consider the following three cases:
    - If the key is less than the middle element, you need to continue to search for the key only in the first half of the array.
    - If the key is equal to the middle element, the search ends with a match.
    - If the key is greater than the middle element, you need to continue to search for the key only in the second half of the array.

  Note: Array should be sorted in ascending order if we want to use Binary Search.

Linear Search

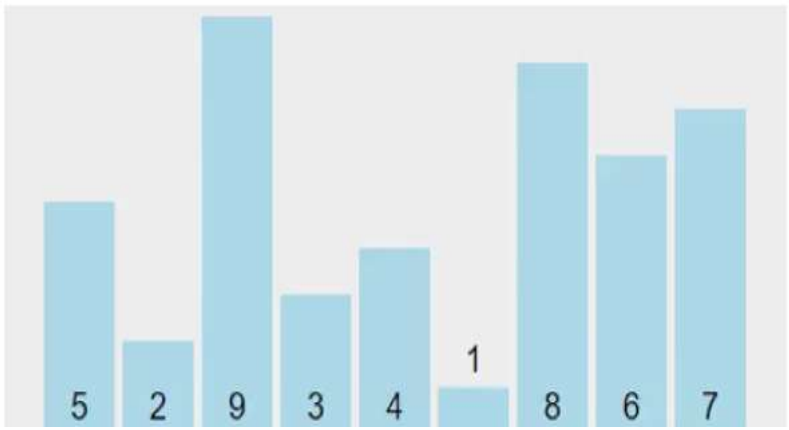
```
import java.util.*;
class LinearSearchDemo{
public static void main(String[] args) {
    int size;
    int a[]={1,2,3,4,5,6,7,8,9};
    int search;
    boolean flag=false;
    Scanner sc=new Scanner(System.in);
    System.out.print("Enter element to search");
    search=sc.nextInt();
       for(int i=0;i<a.length;i++){
           if(a[i]==search){
           System.out.println("element found at "+i+"th index");
           flag=true;
           break;
           }
      }
     if(!flag)
            System.out.println("element NOT found!");
    }
}
```

Output:

```
Enter element to search 6
element found at 5th index
Enter element to search 35
element NOT found!
```

Sorting Array

- Sorting, like searching, is a common task in computer programming. Many different algorithms have been developed for sorting.
- There are many sorting techniques available, we are going to explore selection sort.
- Selection sort
  - finds the smallest number in the list and swaps it with the first element.
  - It then finds the smallest number remaining and swaps it with the second element, and so on, until only a single number remains.



Selection Sort (Example)

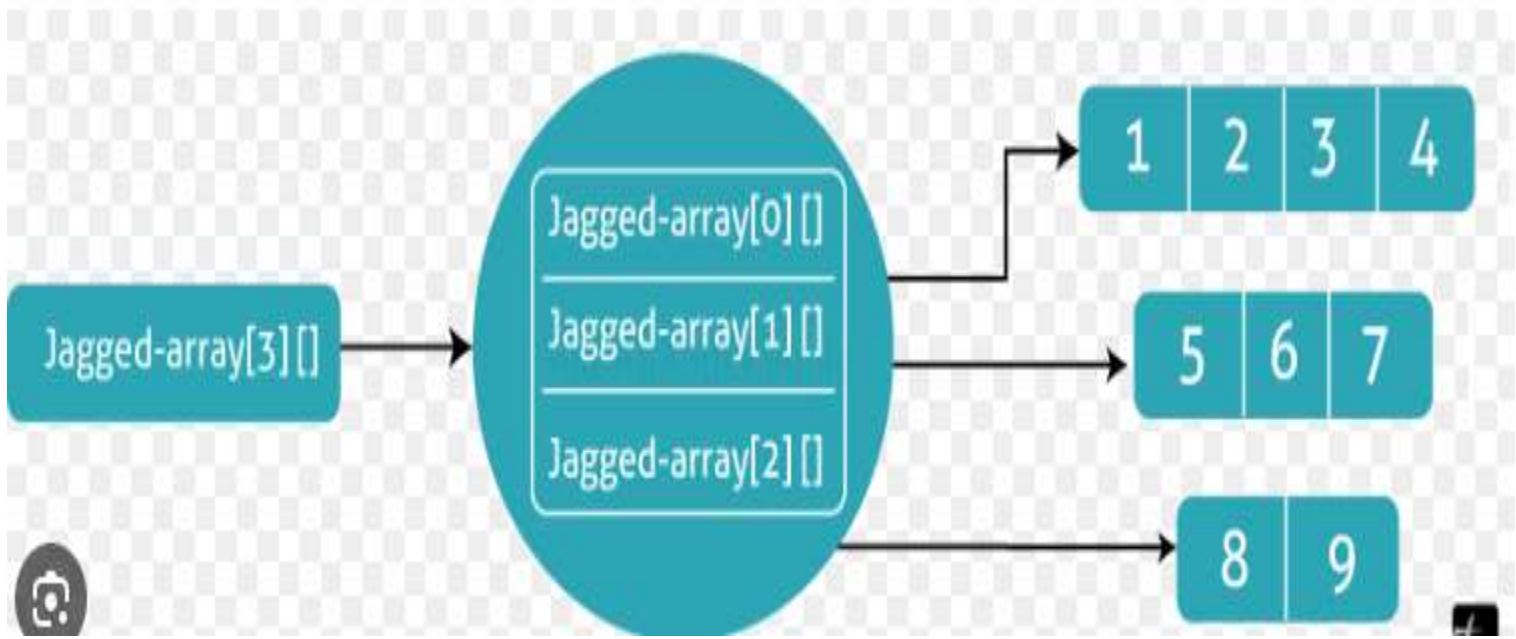
```
. import java.util.*;
. class SelectionSearchDemo{
.  public static void main(String[] args)

.  int a[]={ 5, 2, 9, 3, 4, 1, 8, 6, 7 };

   for (int i = 0; i < a.length - 1; i++) {
   // Find the minimum in the list[i..a.length-1]
     int min = a[i];
     int minIndex = i;
     for (int j = i + 1; j < a.length; j++) {
      if (min > a[j]) {
        min = a[j];
        minIndex = j;
      }
     }//inner for loop j
   // Swap a[i] with a[minIndex]
     if (minIndex != i) {
       a[minIndex] = a[i];
       a[i] = min;
       }
    }//outer for i
    for(int temp: a) { // this is foreach loop
       System.out.print(temp + ", ");
     }
   }//main()
}//class
```

Output:

```
1, 2, 3, 4, 5, 6, 7, 8, 9,
```


Jagged-array[3] []
Jagged-array[0] []
Jagged-array[1] []
Jagged-array[2] []
1 2 3 4
5 6 7
8 9